

Django Module 2 Important

1. How Django ORM Differs from Relational Databases

Django ORM	Relational Database
ORM (Object Relational Mapper) allows interaction with databases using Python code.	Relational databases store data in tables using rows and columns.
Developers work with Python objects and models.	Developers work directly with tables and SQL queries.
Converts Python code into SQL queries automatically.	Requires SQL commands for database operations.
Reduces the need to write complex SQL statements.	SQL must be written manually for operations.
Provides built-in methods like <code>.create()</code> , <code>.filter()</code> , <code>.update()</code> , and <code>.delete()</code> .	Uses SQL commands such as <code>INSERT</code> , <code>SELECT</code> , <code>UPDATE</code> , and <code>DELETE</code> .
Protects against SQL Injection attacks.	Security depends on how SQL queries are written.
Supports multiple databases without changing query code.	Query syntax may vary between database systems.
Maintains database logic in <code>models.py</code> .	Database structure is managed directly in the database.

Django ORM

- Acts as a bridge between Python objects and database tables.
- Model Class → Database Table
- Model Field → Table Column
- Object → Table Row
- Improves code readability and maintainability.

Relational Database

- Stores data in tabular form.
- Uses rows and columns.
- Based on the relational model.
- Uses SQL for data manipulation and retrieval.
- Supports relationships using Primary Keys and Foreign Keys.

2. One-to-Many Relationship in Django

In a One-to-Many relationship, one record in a table can be related to multiple records in another table.

Example:

- One Department has many Students.
- Each Student belongs to only one Department.

Syntax

```
models.ForeignKey(ModelName, on_delete=models.CASCADE)
```

Example

```
from django.db import models

class Department(models.Model):
    name = models.CharField(max_length=100)

class Student(models.Model):
    name = models.CharField(max_length=100)
    department = models.ForeignKey(
        Department,
        on_delete=models.CASCADE
    )
```

Explanation

- **Department** is the parent table.
- **Student** is the child table.
- **ForeignKey** creates the One-to-Many relationship.
- One department can have many students.
- Each student belongs to only one department.
- **on_delete=models.CASCADE** means if a department is deleted, all related student records are also deleted.

Representation

Department

id

name

1
|
|
▼

Student

id

name

department_id

One Department → Many Students.

3.Student Model

```
from django.db import models
```

```
class Student(models.Model):
    student_id = models.AutoField(primary_key=True)
    name = models.CharField(max_length=100)
    email = models.EmailField()
    department = models.CharField(max_length=50)
    phone_number = models.CharField(max_length=15)
    date_of_birth = models.DateField()
```

1) Create at least Three Student Records

```
Student.objects.create(
    name="Arun",
    email="arun@gmail.com",
    department="MCA",
    phone_number="9876543210",
    date_of_birth="2003-05-10"
)
```

```
Student.objects.create(
    name="Anjali",
    email="anjali@gmail.com",
    department="MCA",
    phone_number="9876543211",
    date_of_birth="2004-07-15"
)
```

```
Student.objects.create(
    name="Rahul",
    email="rahul@yahoo.com",
    department="BCA",
    phone_number="9876543212",
    date_of_birth="2001-03-20"
)
```

2) Retrieve and Display All Student Records

```
Student.objects.all()
```

3) Display Students Belonging to MCA Department

```
Student.objects.filter(department="MCA")
```

4) Retrieve Students Whose Names Start with 'A'

```
Student.objects.filter(name__startswith="A")
```

5) Retrieve Students Whose Email Ends with 'gmail.com'

```
Student.objects.filter(email__endswith="gmail.com")
```

6) Update Phone Number Using student_id

```
Student.objects.filter(student_id=1).update(  
    phone_number="999999999"  
)
```

7) Retrieve Students Born After 2002

```
Student.objects.filter(  
    date_of_birth__gt="2002-12-31"  
)
```

8) Delete a Student Record Using student_id

```
Student.objects.filter(student_id=1).delete()
```

QuerySet Methods Used:

- `create()` → Insert records
- `all()` → Retrieve all records
- `filter()` → Retrieve matching records
- `update()` → Modify records
- `delete()` → Remove records

4. i) Adding `__str__`

The `__str__()` method is used to return a readable string representation of an object. It helps display meaningful information instead of showing an object reference.

Example

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField()

    def __str__(self):
        return self.name
```

Purpose:

- Displays the student's name in the Django admin panel.
 - Makes objects easier to identify and read.
 - Improves debugging and data management.
-

ii) Meta Class with `ordering` and `verbose_name`

The `Meta` class is an inner class used to define additional options for a model.

Using `ordering`

`ordering` specifies the default order in which records are retrieved.

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)

    class Meta:
        ordering = ['name']
```

Purpose:

- Automatically sorts records by `name` in ascending order.
- No need to use `order_by()` every time.

Using `verbose_name`

`verbose_name` provides a human-readable singular name for the model.

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)

    class Meta:
        verbose_name = "Student Record"
```

Purpose:

- Displays a user-friendly model name in the admin panel.
- Improves readability for administrators.

Example with Both

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)

    def __str__(self):
        return self.name

    class Meta:
        ordering = ['name']
        verbose_name = "Student Record"
```

- `__str__()` → Returns a readable object name.
- `ordering` → Sets default sorting of records.
- `verbose_name` → Sets a user-friendly model name.

5. Creating Models in Django

A model is a Python class that represents a database table. Each attribute inside the model class represents a field (column) in the database table. Models are defined inside the `models.py` file.

Step 1: Create a Model in `models.py`

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
    email = models.EmailField()
```

Here:

- `Student` → Table name
- `name`, `age`, `email` → Columns in the table

Step 2: Create Migration File

```
python manage.py makemigrations
```

This command:

- Checks changes in `models.py`
- Creates migration files inside the `migrations` folder

Step 3: Apply Migrations

```
python manage.py migrate
```

This command:

- Creates the database table
- Synchronizes the database with the model

Common Field Types

```
from django.db import models
```

```
class Student(models.Model):  
    name = models.CharField(max_length=100)  
    age = models.IntegerField()  
    email = models.EmailField()  
    is_active = models.BooleanField()  
    dob = models.DateField()
```

Field Type	Purpose
<code>CharField</code>	Stores short text
<code>IntegerField</code>	Stores integers
<code>EmailField</code>	Stores email addresses
<code>BooleanField</code>	Stores True/False values
<code>DateField</code>	Stores dates

Example with `__str__()`

```

from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
    email = models.EmailField()

    def __str__(self):
        return self.name

```

The `__str__()` method returns a readable representation of the object.

6. Many-to-Many Relationship in Django

A Many-to-Many relationship means multiple records in one table can be related to multiple records in another table.

Example:

- A Student can study many Subjects.
- A Subject can have many Students.

Syntax

```
models.ManyToManyField(ModelName)
```

Example

```

from django.db import models

class Subject(models.Model):
    name = models.CharField(max_length=100)

class Student(models.Model):
    name = models.CharField(max_length=100)
    subjects = models.ManyToManyField(Subject)

```

Explanation

- One student can have many subjects.
- One subject can have many students.
- `ManyToManyField()` creates the relationship.
- Django automatically creates an intermediate table (`Student_Subject`).
- No need to manually create the third table.

Representation

Student $\longleftrightarrow$ Student_Subject $\longleftrightarrow$ Subject

Many-to-Many with **through**

Used when extra information is required in the relationship.

Syntax

```
models.ManyToManyField(  
    ModelName,  
    through='IntermediateModel'  
)
```

Example

```
from django.db import models  
  
class Student(models.Model):  
    name = models.CharField(max_length=100)  
  
class Subject(models.Model):  
    name = models.CharField(max_length=100)  
  
class Enrollment(models.Model):  
    student = models.ForeignKey(Student, on_delete=models.CASCADE)  
    subject = models.ForeignKey(Subject, on_delete=models.CASCADE)  
    enrollment_date = models.DateField()  
    marks = models.IntegerField()  
  
class Student(models.Model):  
    name = models.CharField(max_length=100)  
    subjects = models.ManyToManyField(  
        Subject,  
        through='Enrollment'  
    )
```

Explanation of **through**

- **Enrollment** acts as the intermediate (bridge) table.
- Stores additional information such as:
 - **enrollment_date**
 - **marks**
- Django will not automatically create the intermediate table because it is created manually.

7. Student and IDCard Models with One-to-One Relationship

models.py

```
from django.db import models

class Student(models.Model):
    student_id = models.AutoField(primary_key=True)
    name = models.CharField(max_length=100)
    email = models.EmailField()
    department = models.CharField(max_length=50)
    phone_number = models.CharField(max_length=15)
    date_of_birth = models.DateField()

    def __str__(self):
        return self.name

class IDCard(models.Model):
    card_id = models.AutoField(primary_key=True)
    card_number = models.CharField(max_length=20)

    issue_date = models.DateField()
    expiry_date = models.DateField()

    student = models.OneToOneField(
        Student,
        on_delete=models.CASCADE
    )

    def __str__(self):
        return self.card_number
```

Explanation

- **Student** contains:
 - **student_id** (Primary Key)
 - **name**
 - **email**
 - **department**
 - **phone_number**
 - **date_of_birth**
- **IDCard** contains:
 - **card_id** (Primary Key)
 - **card_number**

- `issue_date`
 - `expiry_date`
- `OneToOneField()` establishes a **One-to-One relationship** between `Student` and `IDCard`.
- Each student can have only one ID card.
- Each ID card belongs to only one student.
- `on_delete=models.CASCADE` deletes the ID card if the related student is deleted.

Relationship Diagram

Student	IDCard
-----	-----
student_id (PK)	1 ↔ 1 card_id (PK)
name	card_number
email	issue_date
department	expiry_date
phone_number	student (OneToOneField)
date_of_birth	

Create Migrations

```
python manage.py makemigrations
```

Apply Migrations

```
python manage.py migrate
```

This creates both tables and establishes the One-to-One relationship in the database.

8. Model Managers, Custom Manager Methods and Custom Model Methods

1. Model Managers

A Model Manager is the interface through which Django models interact with the database. Every model automatically gets a default manager called `objects`.

Example:

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    department = models.CharField(max_length=50)
```

```
# Using default manager
Student.objects.all()
Student.objects.filter(department="MCA")
```

Uses:

- Retrieve records from the database.
 - Perform filtering and querying operations.
 - Acts as a bridge between models and the database.
-

2. Custom Manager Methods

Custom managers allow developers to define reusable query methods.

Example:

```
from django.db import models

class StudentManager(models.Manager):

    def mca_students(self):
        return self.filter(department="MCA")

class Student(models.Model):
    name = models.CharField(max_length=100)
    department = models.CharField(max_length=50)

    objects = StudentManager()
```

Using the Custom Manager Method

```
Student.objects.mca_students()
```

Benefits:

- Reusable query logic.
 - Cleaner code.
 - Avoids repeating the same filters.
-

3. Custom Model Methods

Custom model methods are functions defined inside a model class to perform operations related to that model.

Example:

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    department = models.CharField(max_length=50)

    def student_info(self):
        return f"{self.name} - {self.department}"
```

Using the Method

```
student = Student.objects.get(id=1)
student.student_info()
```

Output

Arun - MCA

Benefits:

- Encapsulates business logic within the model.
- Improves code reusability.
- Makes models more organized.

Difference

Feature	Model Manager	Custom Manager Method	Custom Model Method
Purpose	Access database records	Create reusable queries	Define object-specific behavior
Defined In	Manager class	Manager class	Model class
Works On	Collection of objects	Collection of objects	Single object
Example	<code>objects.all()</code>	<code>objects.mca_students()</code>	<code>student.student_info()</code>